



มหาวิทยาลัยมหิดล
คณะสิ่งแวดล้อม
และทรัพยากรศาสตร์



Geo-Informatics in
Resource and Environment
Research and Training Center



JavaScript for the Beginners

ความรู้พื้นฐานภาษาจาวาสคริปต์สำหรับผู้เริ่มต้น



Pisut Nakmuenwai, Ph.D.



Outline

- History and Trend of JavaScript
- Syntax
- Operators
- Statement
- Object Oriented Programming (OOP)
- CSS

Statement

Debugging

Conditional

Looping

Error Handle

Function

Declaration



History of JavaScript



Brendan Eich, an American programmer working for Netscape Navigator, created JavaScript in 1994. It was originally known as LiveScript but was renamed JavaScript in 1995 to capitalize on the popularity of Java.

1995-1997

JavaScript was designed to improve website interactivity by enabling developers to add dynamic elements like buttons, sliders, and forms. It also empowered developers to manipulate web page data, such as accessing information from databases and displaying it on the user interface.

Despite its potential, JavaScript didn't gain immediate traction due to its relative infancy and limitations. However, it gradually grew in popularity during the late 1990s and early 2000s. This growth can be attributed to several factors:

- Wider browser support: More web browsers began incorporating JavaScript compatibility.
- Emergence of libraries and tools: Developers created libraries and tools that facilitated JavaScript development.
- Increasing demand for dynamic websites: The web evolved beyond static pages, and JavaScript offered the tools to build interactive experiences.



History of JavaScript

2000s-Present

Today, JavaScript reigns as the most popular client-side programming language globally. It's used to build websites, web applications, and even mobile applications.

Key milestones in JavaScript's evolution:

1997: Became an ECMA standard (ECMAScript)

2005: Prototype-based inheritance introduced

2009: Strict mode implemented

2015: ES6 (ECMAScript 2015) released with significant changes like modules and classes

2023: Latest version, ES2023, released with features like optional chaining and nullish coalescing operator



Java vs JavaScript



JavaScript และ Java เป็นภาษาโปรแกรมที่แตกต่างกันโดยสิ้นเชิง โดยที่ JavaScript ถูกสร้างขึ้นโดย Brendan Eich ของ Netscape Communications Corporation โดย Eich เริ่มต้นการพัฒนา JavaScript เริ่มพัฒนาปี 1995 และเริ่มใช้งาน 4 ธ.ค. 1995 เพื่อเพิ่มความสามารถในการโต้ตอบให้กับหน้าเว็บในยุคแรกๆ ในขณะนั้น ภาษาโปรแกรมที่ได้รับความนิยมมากที่สุดคือ Java ที่พัฒนาโดย James Gosling ของ Sun Microsystems เริ่มพัฒนา มี.ย. 1991 เริ่มใช้งาน 23 พ.ค. 1995 และ v1.0 เมื่อ 1996 เป็นภาษาโปรแกรมเชิงโอเปอเรชันที่จำเป็นต้องคอมไพล์ก่อนจึงจะสามารถทำงานได้ Eich ต้องการภาษาโปรแกรมที่ง่ายกว่าสำหรับนักพัฒนาเว็บที่จะใช้ และเขาต้องการภาษาโปรแกรมที่สามารถทำงานได้โดยตรงในเบราว์เซอร์โดยไม่ต้องคอมไพล์ก่อน

Eich เลือกชื่อ "JavaScript" เพื่อเป็นเกียรติแก่ Java เนื่องจากทั้งสองภาษามีเป้าหมายที่จะทำให้การพัฒนาเว็บง่ายขึ้น อย่างไรก็ตาม Eich เน้นย้ำว่า JavaScript ไม่ได้เป็นภาษาโปรแกรมเดียวกับ Java และทั้งสองภาษาไม่มีความสัมพันธ์กัน

Feature	Java	JavaScript
Type	General-purpose, object-oriented programming language	Scripting language
Primary Use	Back-end development, mobile app development, enterprise applications	Front-end web development, interactive web pages, server-side development (with Node.js)
Execution	Compiled bytecode runs on Java Virtual Machine (JVM)	Interpreted directly by web browsers
Typing	Statically typed	Dynamically typed
Object-Oriented	Yes	Yes, but with some prototypal inheritance differences
Popularity	Very popular, widely used in various industries	Extremely popular, essential for web development
Learning Curve	Moderate to steep	Relatively easy, especially for beginners with HTML/CSS knowledge
Examples of Applications	Android apps, desktop applications, enterprise software, financial systems	Web applications, online games, interactive websites, browser extensions

Web Developer

HTML



Hypertext Markup Language

Create the structure

- Controls the layout of the content
- Provides structure for the web page design
- Fundamental building block of any web page

CSS



Cascading Style Sheet

Stylize the website

- Applies style to the web page elements
- Targets various screen sizes to make web pages responsive
- Primarily handles the “look and feel” of the web page

JS



JavaScript

Increase interactivity

- Adds interactivity to a web page
- Handles complex functions and features
- Programmatic code which enhances functionality

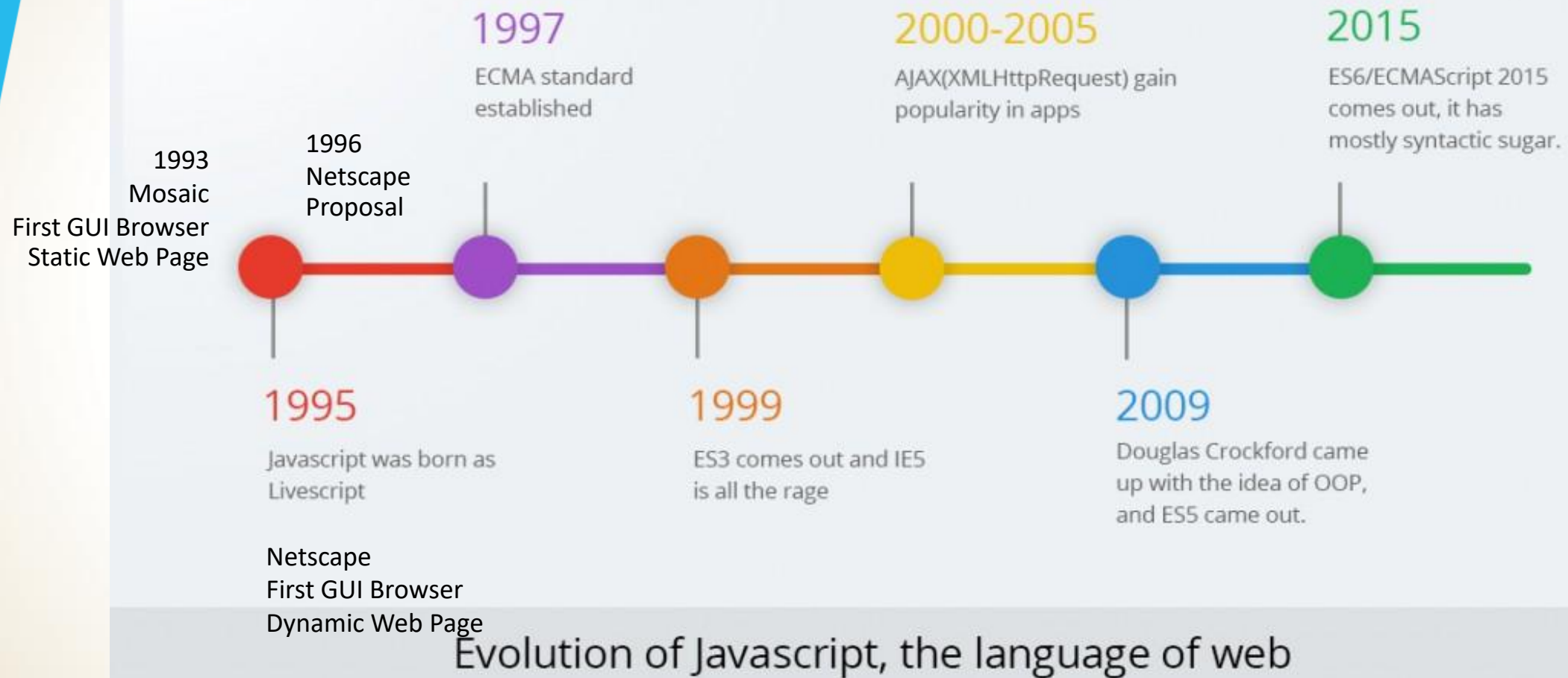
```
<html>
  <head>
    ....
  </head>
  <body>
    ....
  </body>
</html>
```

- Inline - by using the style attribute inside HTML elements (style=“...” and class=“...”)
- Internal - by using a `<style>` element in the `<head>` section
- External - by using a `<link>` element to link to an external CSS file

- Inline - by using the event attribute inside HTML elements (onload=“...” , onclick=“...”)
- Internal - by using a `<script>` element in the `<head>` and `<body>` section
- External - by using a `<script src=“file.js”>` element to link to an external JavaScript file

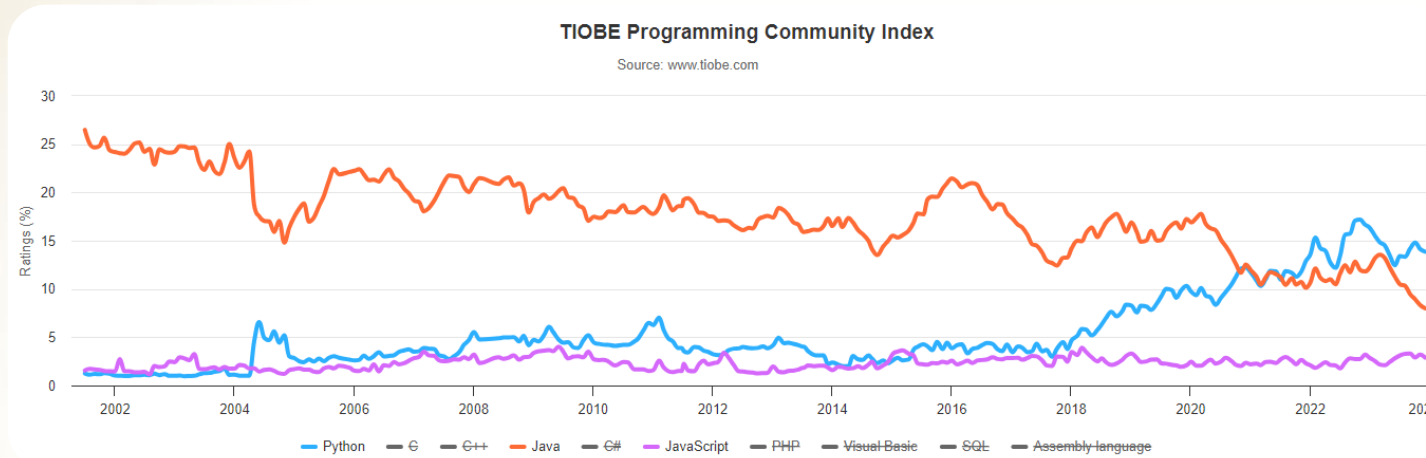


Note: **The GEE editor supports most features of ECMAScript 5 (ES5)**, a standardized specification of the JavaScript language. Language features introduced in ECMAScript 6 (ES6) and above are not supported at this time.





Future of JavaScript



Developer Salary

R developer	\$77,776	
Java/JavaScript developer	\$90,632	
HTML developer	\$68,165	
Ruby developer	\$88,203	
PHP developer	\$77,822	
C developer	\$89,013	
Python developer	\$76,526	

With its versatility and continuous evolution, JavaScript is poised to remain a dominant force in web development. Its future holds exciting possibilities, including:

Widespread adoption in emerging technologies: JavaScript is expected to play a key role in areas like blockchain and the metaverse. Further integration with other languages and frameworks: JavaScript's ability to integrate with other technologies will likely increase its reach. Continued evolution of the language itself: New features and improvements will likely be added to the language to support future needs.



JavaScript ECMAScript Version

Edition	Date published	Name	Changes from prior edition	Editor
1	June 1997		First edition based on JavaScript 1.1 as implemented in Netscape Navigator 3.0. ^[1]	Guy L. Steele Jr.
2	June 1998		Editorial changes to keep the specification fully aligned with ISO/IEC 16262:1998.	Mike Cowlishaw
3	December 1999		Based on JavaScript 1.2 as implemented in Netscape Navigator 4.0. ^[2] Added regular expressions , better string handling, new control statements, try/catch exception handling, tighter definition of errors, formatting for numeric output, and other enhancements	Mike Cowlishaw
4	<i>Abandoned</i> (last draft 30 June 2003)		Fourth Edition was abandoned, due to political differences concerning language complexity. Many features proposed for the Fourth Edition have been completely dropped; some were incorporated into the sixth edition.	
5	December 2009		Adds "strict mode", a subset intended to provide more thorough error checking and avoid error-prone constructs. Clarifies many ambiguities in the 3rd edition specification, and accommodates behavior of real-world implementations that differed consistently from that specification. Adds some new features, such as getters and setters, library support for JSON , and more complete reflection on object properties. ^[3]	Pratap Lakshman, Allen Wirfs-Brock
5.1	June 2011		Changes to keep the specification fully aligned with ISO/IEC 16262:2011.	Pratap Lakshman, Allen Wirfs-Brock

Edition	Date published	Name	Changes from prior edition	Editor
6	June 2015 ^[4]	ECMAScript 2015 (ES2015)	See 6th Edition – ECMAScript 2015	Allen Wirfs-Brock
7	June 2016 ^[5]	ECMAScript 2016 (ES2016)	See 7th Edition – ECMAScript 2016	Brian Terlson
8	June 2017 ^[6]	ECMAScript 2017 (ES2017)	See 8th Edition – ECMAScript 2017	Brian Terlson
9	June 2018 ^[7]	ECMAScript 2018 (ES2018)	See 9th Edition – ECMAScript 2018	Brian Terlson
10	June 2019 ^[8]	ECMAScript 2019 (ES2019)	See 10th Edition – ECMAScript 2019	Brian Terlson, Bradley Farias, Jordan Harband
11	June 2020 ^[9]	ECMAScript 2020 (ES2020)	See 11th Edition – ECMAScript 2020	Jordan Harband, Kevin Smith
12	June 2021 ^[10]	ECMAScript 2021 (ES2021)	See 12th Edition – ECMAScript 2021	Jordan Harband, Shu-yu Guo, Michael Ficarra, Kevin Gibbons
13	June 2022 ^[11]	ECMAScript 2022 (ES2022)	See 13th Edition – ECMAScript 2022	Shu-yu Guo, Michael Ficarra, Kevin Gibbons
14	June 2023	ECMAScript 2023 (ES2023)	See 14th Edition – ECMAScript 2023	Shu-yu Guo, Michael Ficarra, Kevin Gibbons



ECMA5 vs ECMA6

Feature	ES5 ECMAScript 5	ES6 ECMAScript 6
Functions:	Function keyword required	Arrow functions introduced, offering a more concise syntax
Classes:	Prototype-based inheritance	Class syntax introduced, similar to other object-oriented languages
Modules:	No official module system	Module system introduced, enabling cleaner and more organized code
Variables:	var keyword used for variable declaration	let and const keywords introduced for block-scoped and constant variables
Template literals:	String concatenation required	Template literals introduced, providing easier string interpolation
Spread and rest operators:	N/A	Spread (...) and rest (...) operators introduced for convenient element manipulation
Default parameter values:	Function arguments can't have default values	Function arguments can have default values, improving code clarity
Promise API:	N/A	Promise API introduced for asynchronous programming



ECMA5 vs ECMA6

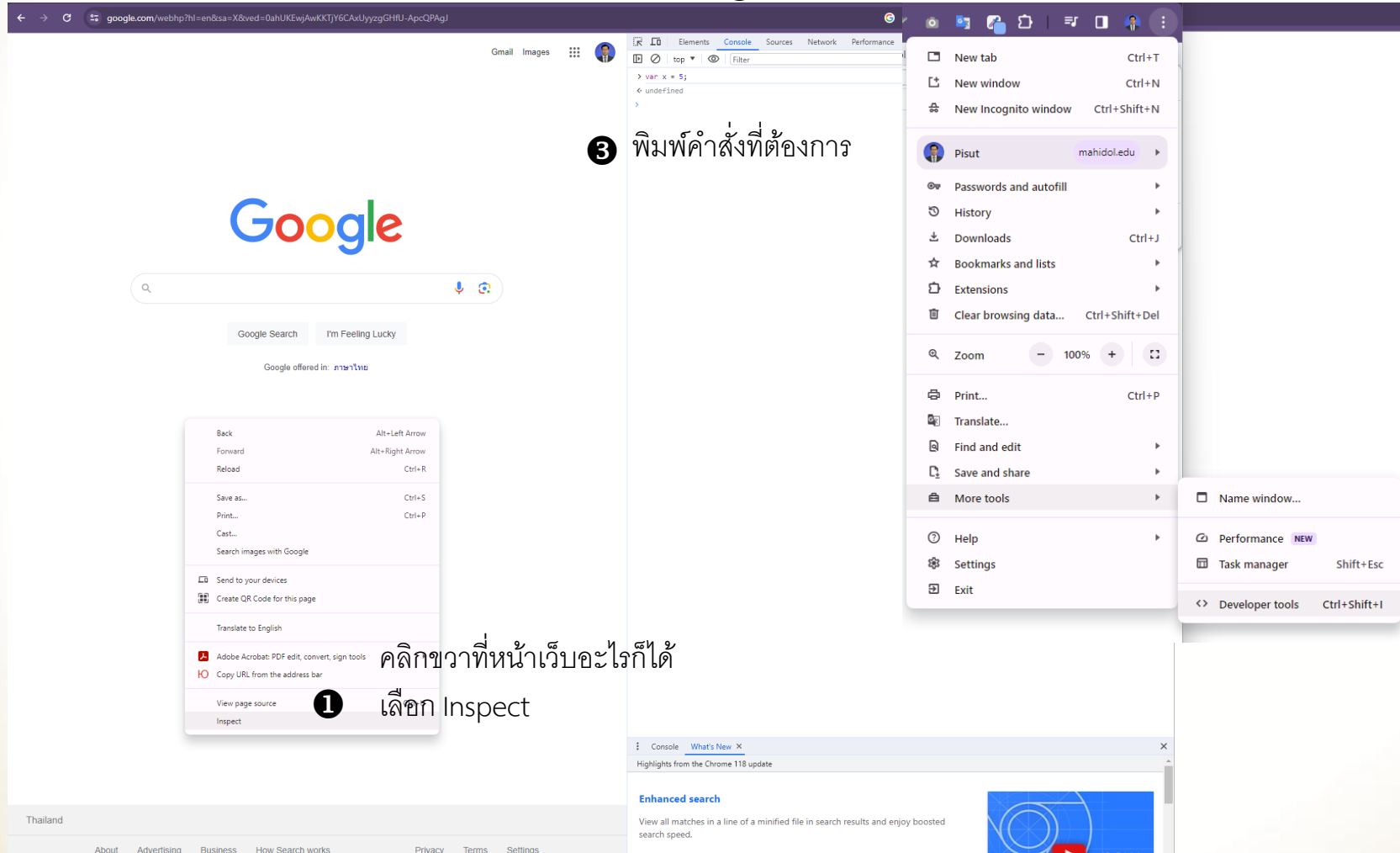
Feature	ES5 ECMAScript 5	ES6 ECMAScript 6
Functions:	<code>function () { ... }</code>	<code>() => { ... }</code>
Classes:	<code>function A(x) { ... }; var a = new A(1);</code>	<code>Class A(x) { ... }; var a = new A(1);</code>
Modules:	-	<code>export function A () { return 1; }</code> <code>import { A } from ".../fileA.js"</code>
Variables:	<code>var a = 1;</code>	<code>let a = 1;</code> <code>const b = 2;</code>
Template literals:	<code>var x = 'number' + a;</code>	<code>let x = `number \${a}`;</code> <code>let backtick = `multiple line string`;</code>
Spread and rest operators:	N/A	<code>let a = [1, 2, 3]; let b = [4, 5, 6];</code> <code>let c = [...a, ...b]; // [1, 2, 3, 4, 5, 6];</code>
Default parameter values:	<code>function x(a, b) { return a + (b ? b : 2); }</code>	<code>function x(a, b = 2) { return a + b; }</code>
Promise API:	N/A	<code>let fetchData = new Promise((resolve, reject) => { ... });</code> <code>fetchData("https://example.com/data")</code> <code>.then(data => console.log(data))</code> <code>.catch(error => console.error(error));</code>

ECMA8 using `async/await` keywords

ทดลองเขียน JavaScript ได้ที่ไหนบ้าง

ใช้งานกับ Web Browser เช่น Google Chrome

2 เลือก tab console



3 พิมพ์คำสั่งที่ต้องการ

คลิกขวาที่หน้าเว็บอะไรก็ได้

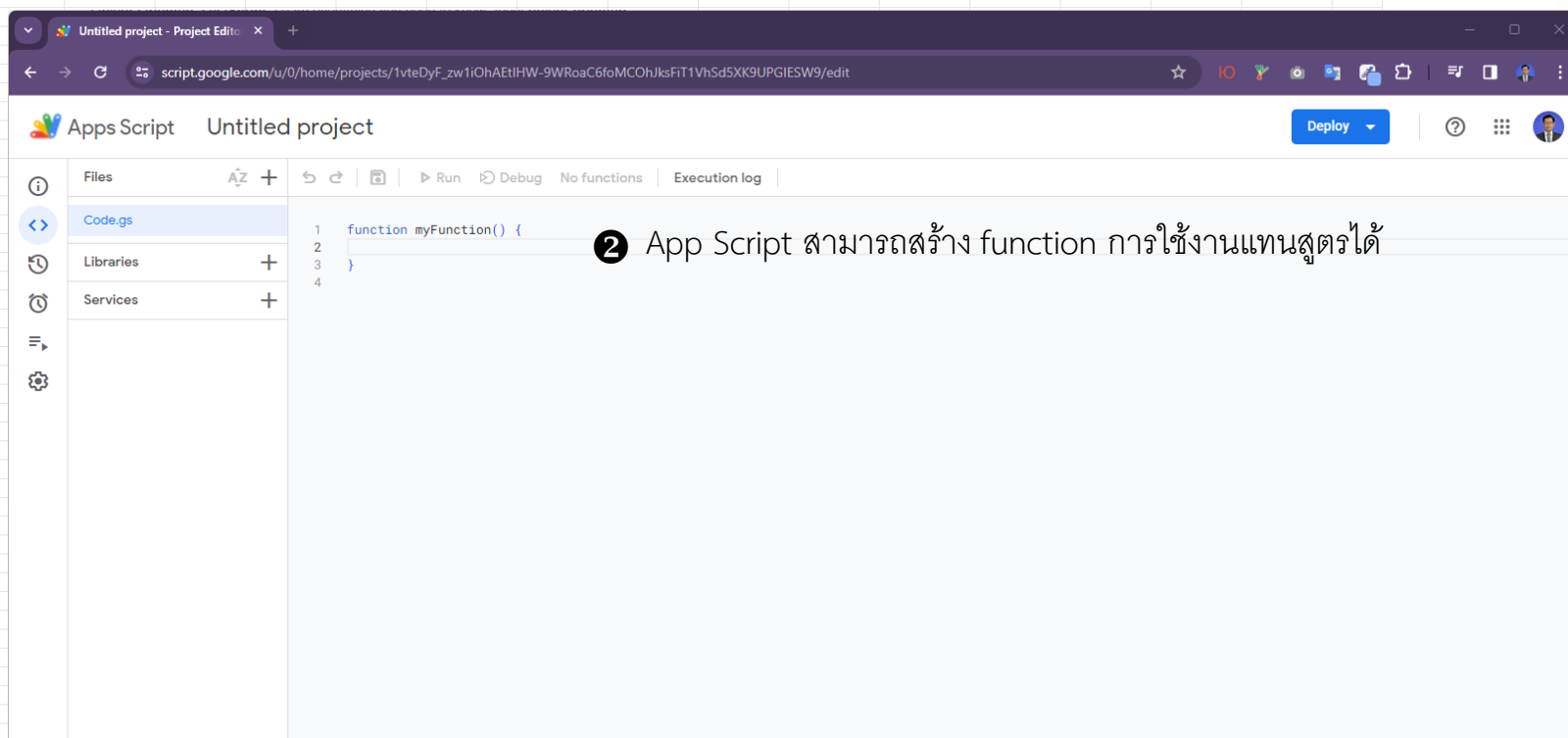
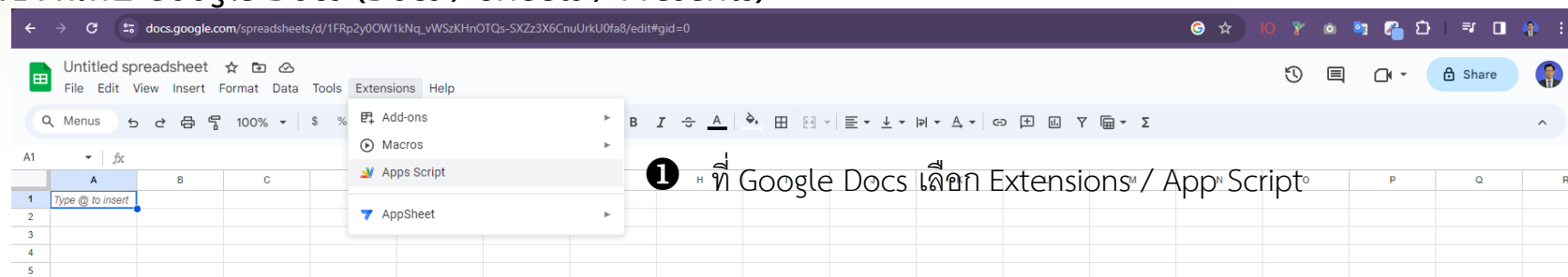
1 เลือก Inspect

หรือเลือก เมนู

1 More tools / Developer tools

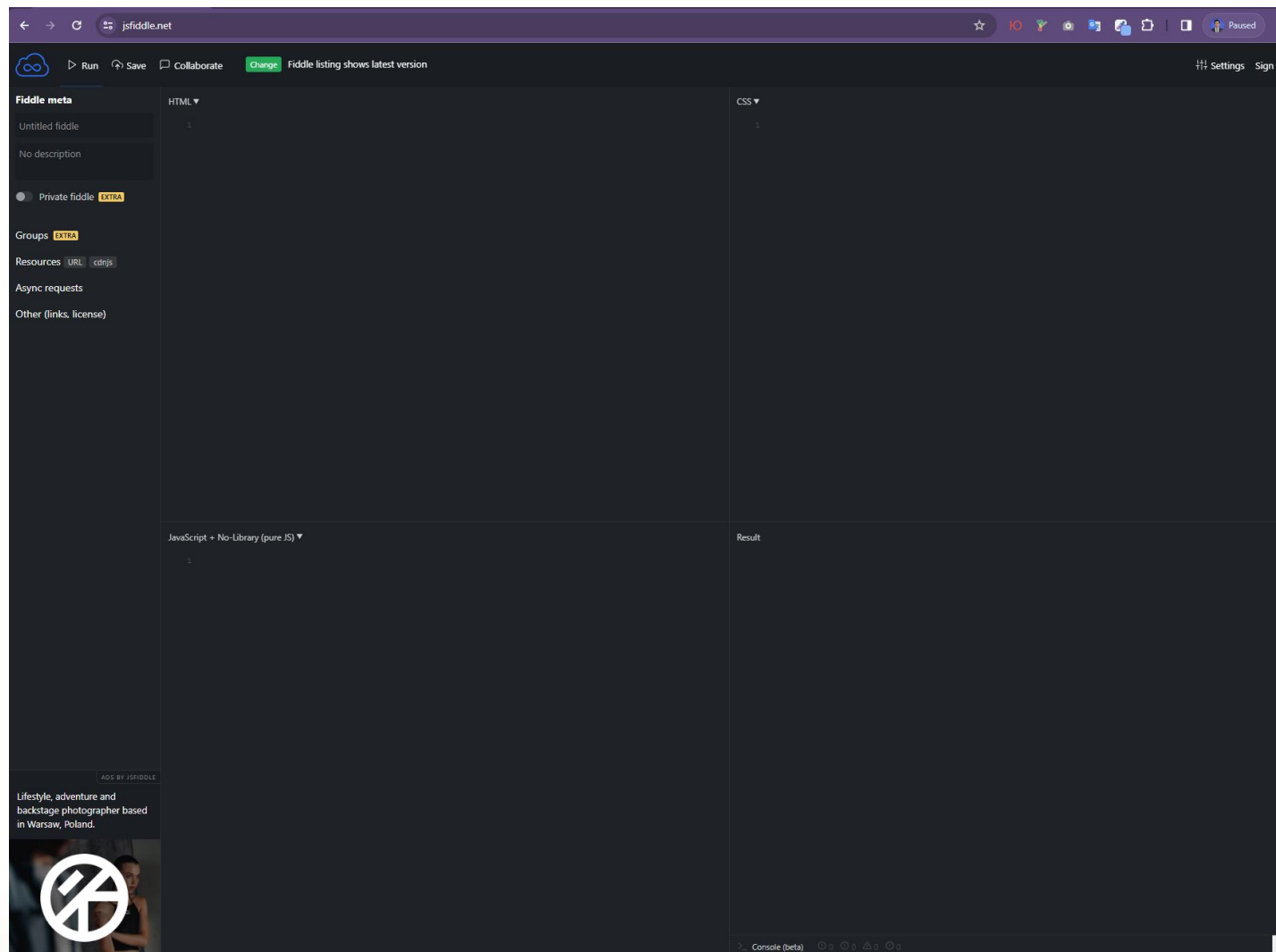
ทดลองเขียน JavaScript ได้ที่ไหนบ้าง

ใช้งานกับ Google Docs (Docs / Sheets / Presents)





ทดลองเขียน JavaScript ได้ที่ไหนบ้าง





ทดลองเขียน JavaScript ได้ที่ไหนบ้าง



```
File Edit Selection View Go ... Search
JS console.js x
D: > Temp > JS console.js
1 console.log('test message')

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Program Files\nodejs\node.exe .\console.js
test message console.js:1

Ln 1, Col 28 Spaces: 4 UTF-8 CRLF {} JavaScript
```



JavaScript ECMA 2009 (ES5) Practices

W3Schools (basic)

https://www.w3schools.com/js/js_es5.asp

W3Schools (back-end)

<https://www.w3schools.com/nodejs/>

W3Schools (front-end)

https://www.w3schools.com/js/js_htmlDOM.asp

Programiz

<https://www.programiz.com/javascript/get-started>

MDN

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

Syntax



คุณสมบัติที่ควรทราบของ JavaScript

- เดิมเป็นภาษาในกลุ่ม Script ที่ต้องแปลผลอยู่บน Application หลัก
แต่ปัจจุบันจัดเป็นภาษาคอมพิวเตอร์ที่สามารถทำงานเป็นเอกเทศได้ด้วยตัวแปลภาษา V8 เช่น Node.JS
- เป็นภาษาคอมพิวเตอร์แบบ Interpreter ที่จะแปลตีความเป็นภาษาเครื่องทีละบรรทัดแล้วทำงาน ต่างจากภาษาคอมพิวเตอร์แบบ Compiler ที่จะแปลตีความเป็นภาษาเครื่องทั้งหมดจึงจะทำงานได้ (GEE Code Editor ต้องแปลทั้งหมดถึงทำงานได้)
- อยู่ในตระกูล C ที่อักษรตัวเล็กและตัวใหญ่จะแตกต่างกัน (case sensitive)
- ไม่จำเป็นต้องมีการประกาศตัวแปร (แต่แนะนำให้ประกาศตัวแปรด้วย var หรือ let และ const)
- ไม่มีโครงสร้าง ย่อหน้า การแบ่งบรรทัด และวรรคตอน การแบ่งชุดคำสั่งจะคั่นด้วย ;
- เป็น Object Oriented Programming (OOP) และ event driver
- รองรับการเขียนแบบลูกโซ่ (chaining) เช่น a.b.c.d;
- JavaScript ES6 รองรับมาตรฐานสมัยใหม่ที่มีในภาษาอื่นๆ เช่น class, promise (แต่ GEE Code Editor เป็น ES5)



Comment

In JavaScript, comments are annotations you add to your code to make it clearer and easier to understand. They are ignored by the JavaScript engine and don't affect how your program runs.

Single-line Comments

`// comment`
code after `//` in the same line will not be interpreted

Multi-line Comments

`/* multiple line
comment */`
code between `/*...*/` will not be interpreted

Toggleable-Block Comment

`/**`
`remove the first /
to toggle this block
able to be interpreted or
not`
`/**/`





String & Quotation

In JavaScript, the term "quotation" can refer to two different things:

1. String delimiters:

These are the characters that are used to define the beginning and end of a string literal. The most common quotation marks used in JavaScript are:

- Single quotes (') e.g. `'Text'`
- Double quotes (") e.g. `"Text"`
- Backticks (`) e.g. ``Text``

Multiple-line string

ES5 ECMA5

```
var a = 'this is \nmultiline text';
```

ES6 ECMA6

```
let a = `this is \nmultiline text`;
```

2. Quoted text:

This refers to the actual text enclosed within the quotation marks. This can be anything from a single word to a paragraph or even multiple lines of code. Quoted text is often used to represent:

In appropriate cases, we can use a combination of techniques to avoid using escape characters when writing these characters in a text.

Quoted text with escape character

```
let quote = 'She said, \'I\'ll be back\'';
```

Quoted text with multiple string delimiters

```
let quote = "She said, 'I'll be back'";
```

Escape Character

ใช้แทนข้อมูลอักขระบางตัวไม่สามารถพิมพ์ได้ทางคีย์บอร์ด เช่น tab (`\t`) และ enter (`\n`) รวมถึง quotation ใน quotation (`\"`)



Variable

In simple terms, a variable in JavaScript is like a named container that stores information. You can use variables to hold different types of data, like numbers, text, Booleans (true/false), arrays, or even objects.

- Declaration: You declare a variable using a keyword like `let`, `const`, or `var` followed by the variable name. For example:

```
let name = "John";  
const age = 30;  
var isLoggedIn = true;
```

- Assignment: You can assign a value to a variable using the `=` operator. For example:

```
let age = 30;  
age = 31; // Update the value of the variable
```

- Scope: The scope of a variable determines where it can be accessed within your code. Variables declared with `let` and `const` have block scope, meaning they are only accessible within the block they are declared in. Variables declared with `var` have function and global scope, meaning they are accessible throughout your code.

JavaScript has 8 Datatypes

1. String
2. Number
3. BigInt
4. Boolean
5. Undefined
6. Null
7. Symbol
8. Object

Native Object Data Type

1. Object
2. Array
3. Date



Data Types

Everything in JavaScript is Object, We usually avoid to write a full form to declare a variable

Data Types

1. String
2. Number
3. BigInt
4. Boolean
5. Undefined
6. Null
7. Symbol
8. Object

Native Object Data Type

1. Object (Dictionary)
2. Array
3. Date

Full form declaration

```
var x = String('1');  
var x = Number(1);  
var x = BigInt(1);  
var x = Boolean(1);  
var x = undefined;  
var x = null;  
var x = Symbol();  
var x = new Object();  
Native Object Data Type  
var x = new Object();  
var x = new Array();  
var x = new Date(2023,0,1);
```

Short form declaration

```
var x = '1';  
var x = 1;  
var x = 1n ;  
var x = true;  
var x = undefined; //del x;  
var x = null;  
var x = Symbol();  
var x = {};  
Native Object Data Type  
var x = { };  
var x = [ ];  
var x = new Date();
```



Primitive & Build-in Object

A primitive (primitive value, primitive data type) is data that is not an object and has no methods or properties. There are 7 primitive data types (string, number, bigint, boolean, undefined, null, symbol)

primitive number is not the same as Number Object

```
var x = 5;
var y = Number(5);
var z = new Number(5);

console.log(x == y); // true
console.log(x === y); //true

console.log(x == z); //true
console.log(x === z); //false
```

An object in an ECMAScript implementation whose semantics are entirely determined by this specification as opposed to the host environment. Native objects are additionally known as Built-in Objects.

- Value properties
- Function properties
- Fundamental objects
- Error objects
- Numbers and dates
- Text processing
- Indexed collections
- Keyed collections
- Structured data
- Managing memory
- Control abstraction objects
- Reflection



Build-in Object

Fundamental objects

- [Object](#)
- [Function](#)
- [Boolean](#)
- [Symbol](#)

Text processing

- [String](#)
- [RegExp](#)

Numbers and dates

- [Number](#)
- [BigInt](#)
- [Math](#)
- [Date](#)

Value properties

- [globalThis](#)
- [Infinity](#)
- [NaN](#)
- [undefined](#)

Function properties

- [eval\(\)](#)
- [isFinite\(\)](#)
- [isNaN\(\)](#)
- [parseFloat\(\)](#)
- [parseInt\(\)](#)
- [decodeURI\(\)](#)
- [decodeURIComponent\(\)](#)
- [encodeURI\(\)](#)
- [encodeURIComponent\(\)](#)

Error objects

- [Error](#)
- [AggregateError](#)
- [EvalError](#)
- [RangeError](#)
- [ReferenceError](#)
- [SyntaxError](#)
- [TypeError](#)
- [URIError](#)

Indexed collections

- [Array](#)
- [Int8Array](#)
- [Uint8Array](#)
- [Uint8ClampedArray](#)
- [Int16Array](#)
- [Uint16Array](#)
- [Int32Array](#)
- [Uint32Array](#)
- [BigInt64Array](#)
- [BigUint64Array](#)
- [Float32Array](#)
- [Float64Array](#)

Keyed collections

- [Map](#)
- [Set](#)
- [WeakMap](#)
- [WeakSet](#)

Operators



JavaScript Operators

JavaScript offers a diverse range of operators to manipulate data, perform calculations, and control program flow.

Assignment Operators

Oper	Example	Same As	Result
=	x = y	x = y	x = 5
+=	x += y	x = x + y	x = 15
-=	x -= y	x = x - y	x = 5
*=	x *= y	x = x * y	x = 50
/=	x /= y	x = x / y	x = 2
%=	x %= y	x = x % y	x = 0
:	x: 45	size.x = 45	x = 45

ES6

Arithmetic Operators

Oper	Name	Example	Results
+	Addition	x = y + 2	y=5, x=7
-	Subtraction	x=y-2	y=5, x=3
*	Multiplication	x=y*2	y=5, x=10
**	Exponentiation ES2016	x=y**2	y=5, x=25
/	Division	x = y / 2	y=5, x=2.5
%	Remainder	x = y % 2	y=5, x=1
++	Pre increment	x = ++y	y=6, x=6
++	Post increment	x = y++	y=6, x=5
--	Pre decrement	x = --y	y=4, x=4
--	Post decrement	x = y--	y=4, x=5



JavaScript Operators

String Operators

Oper	Example	t1	t2	t3
+	t3 = t1 + t2	"Good "	"Morning"	"Good Morning"
+=	t1 += t2	"Good Morning"	"Morning"	

Conditional (Ternary) Operator

Syntax	Example
(condition) ? x : y	(z < 18) ? x : y

false , 0 , null, undefined is false condition, the other is true.

Logical Operators

Oper	Name	Example
&&	AND	(x < 10 && y > 1) is true
	OR	(x === 5 y === 5) is false
!	NOT	!(x === y) is true

Comparison Operators

Oper	Name	Comparing	Returns
==	equal to	x == 8	false
==	equal to	x == 5	true
===	equal value and type	x === "5"	false
===	equal value and type	x === 5	true
!=	not equal	x != 8	true
!=	not equal value or type	x != "5"	true
!=	not equal value or type	x != 5	false
>	greater than	x > 8	false
<	less than	x < 8	true
>=	greater or equal to	x >= 8	false
<=	less or equal to	x <= 8	true

ES6 The Nullish Coalescing Operator (??)
The Optional Chaining Operator (?.)



JavaScript Operators

Bitwise Operators

Very useful for cloud masking by PixelQA Band

Oper	Name	Example	Same as	Result	Decimal
&	AND	x = 5 & 1	0101 & 0001	0001	1
	OR	x = 5 1	0101 0001	0101	5
~	NOT	x = ~ 5	~0101	1010	10
^	XOR	x = 5 ^ 1	0101 ^ 0001	0100	4
<<	Left shift	x = 5 << 1	0101 << 1	1010	10
>>	Right shift	x = 5 >> 1	0101 >> 1	0010	2
>>>	Unsigned right	x = 5 >>> 1	0101 >>> 1	0010	2



JavaScript Operators

The typeof Operator

```
typeof "John"           // 'string'
typeof 3.14              // 'number'
typeof NaN               // 'number'
typeof false             // 'Boolean'
typeof [1, 2, 3, 4]      // 'object'
typeof {name:'John', age:34} // 'object'
typeof new Date()        // 'object'
typeof function () {}    // 'function'
typeof myCar              // 'undefined'
typeof null              // 'object'
```

The instanceof Operator

```
"John" instanceof String      // true
3.14 instanceof Number        // false
NaN instanceof Number         // false
false instanceof Boolean      // false
[1, 2, 3, 4] instanceof Array // true
({name:'John', age:34}) instanceof Object // true
(new Date()) instanceof Object // true
(function () {}) instanceof Object // true
(function () {}) instanceof Function // true
myCar instanceof undefined    // error
null instanceof Object        // false
```

JavaScript `Object.constructor.name` will return object type name
GEE `Object.getName()` will return object type name

Statement



Debugging

Conditional

Looping

Error Handle

Function

Declaration

Statement

Statement	Description
<u>break</u>	Exits a switch or a loop
<u>class</u>	Declares a class
ES6 <u>const</u>	Declares a variable with a constant value
<u>continue</u>	Breaks one iteration (in the loop) if a specified condition occurs, and continues with the next iteration in the loop
<u>debugger</u>	Stops the execution of JavaScript, and calls (if available) the debugging function
<u>do ... while</u>	Executes a block of statements and repeats the block while a condition is true
<u>for</u>	Loops through a block of code a number of times
<u>for ... in</u>	Loops through the properties of an object
<u>for ... of</u>	Loops through the values of an iterable object
<u>function</u>	Declares a function
<u>if ... else ... else if</u>	Marks a block of statements to be executed depending on a condition
ES6 <u>let</u>	Declares a variable
<u>return</u>	Stops the execution of a function and returns a value from that function
<u>switch</u>	Marks a block of statements to be executed depending on different cases
<u>throw</u>	Throws (generates) an error
<u>try ... catch ... finally</u>	Marks the block of statements to be executed when an error occurs in a try block, and implements error handling
<u>var</u>	Declares a variable
<u>while</u>	Marks a block of statements to be executed while a condition is true

Debugging

Browser

Printer

`print()`

Source

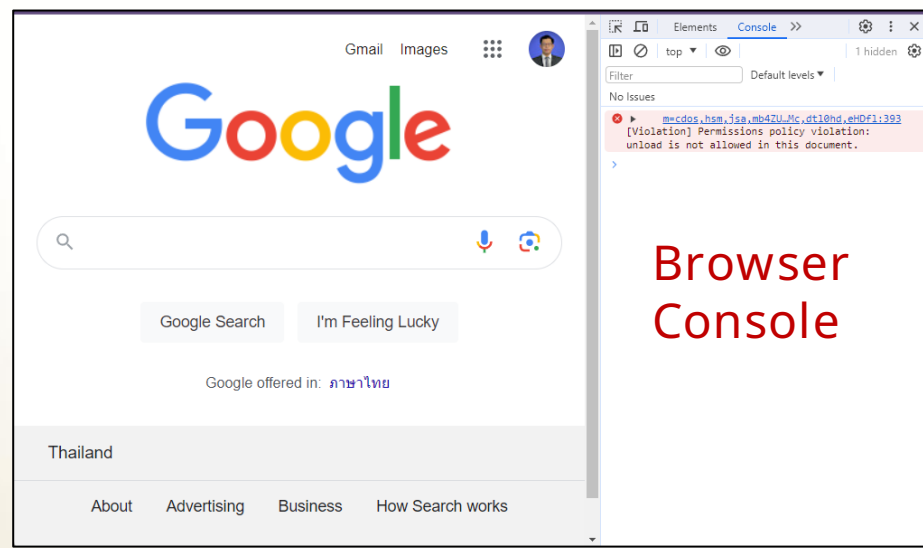
Breakpoint

Popup

`window.alert('message')`

Console

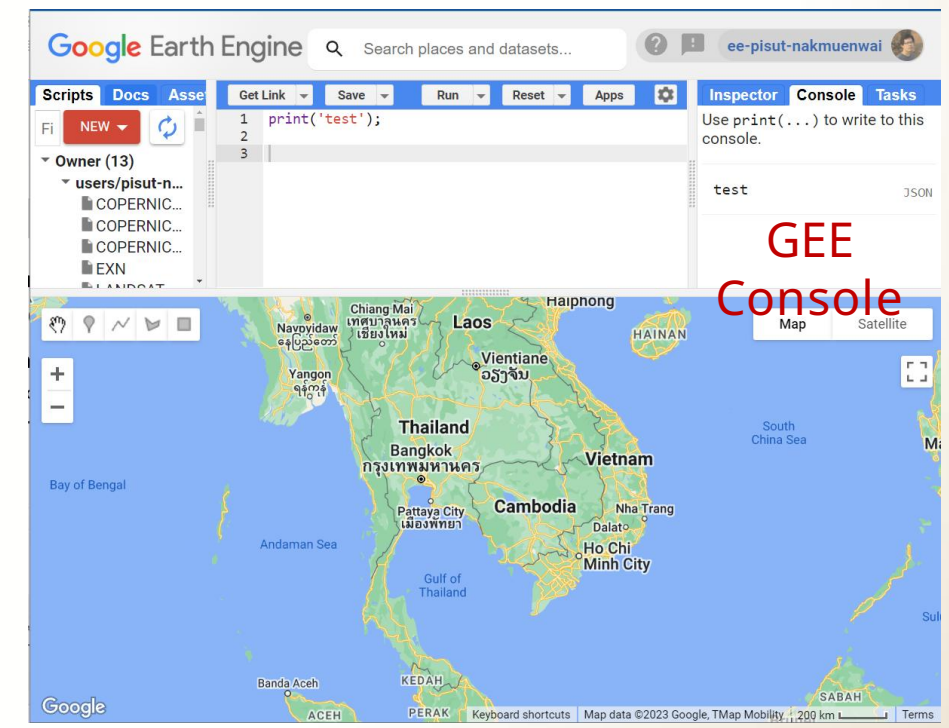
`console.log(object)`



GEE

Console

`print(object)`





Error Handle

Error handling is a crucial aspect of any program, especially in JavaScript, where unexpected situations can arise frequently. It allows your code to gracefully handle errors, prevent crashes, and provide informative feedback to users. Here's a breakdown of error handling in JavaScript:

Try-Catch / Finally

ใช้ดักจับความผิดปกติของการทำงานเพื่อจัดการกับความผิดปกตินั้น

```
try {  
  var x = y;  
}catch {  
  console.log("has error");  
}
```

```
try {  
  var x = y;  
}catch (error){  
  console.log(error.message);  
  //y is not defined  
}
```

```
try {  
  var x = y;  
}catch (error){  
  console.log(error.message);  
  //y is not defined  
}finally {  
  console.log("finish try");  
}
```

Throw

ส่งความผิดพลาดแบบกำหนดเอง

```
function Test(a){  
  if (a=="")  
    throw "a should not be empty";  
}
```


Looping

For

ES5 ทำคำสั่งใน {...} ไปจนกว่าค่าในตัวแปรจะไม่ตรงกับเงื่อนไข โดยให้เพิ่มค่าในแต่ละรอบตามที่กำหนด

```
for (var i=0; i<5; i++){ console.log(i) }  
// 0 1 2 3 4
```

ES5 ทำคำสั่งใน {...} จนจำนวนรอบเท่ากับจำนวนรายการ โดยส่งค่ารอบที่ไปยังตัวแปรที่กำหนด

```
for (var i in ['a', 'b', 'c']){ console.log(i) }  
// 0 1 2 3 4
```

ES6 ทำคำสั่งใน {...} จนจำนวนรอบเท่ากับจำนวนรายการ โดยส่งค่า array value หรือ dictionary key ไปยังตัวแปรที่กำหนด

```
for (var i of ['a', 'b', 'c']){ console.log(i) }  
// 'a' 'b' 'c'
```

While

ทำคำสั่งใน {...} ในขณะที่เงื่อนไขเป็นจริง มีความยืดหยุ่นสูงและผิดได้ง่ายจนบางครั้งไม่สามารถออกจาก loop ได้

```
do {...} while (condition)
```

Break/Continue

break ใช้สำหรับออกจาก loop ณ ตำแหน่งนั้นโดยไม่ต้องทำงานอีก
continue ใช้สำหรับข้ามการทำงานในส่วนที่เหลือนั้นแล้วไปเริ่มใน loop ถัดไป

ไม่แนะนำให้ใช้ for while กับข้อมูล BigData
แนะนำให้ใช้ **map** และ **forEach** ซึ่งจะกล่าวถึงในหัวข้อถัดไป



Conditional

If else

การพิสูจน์เงื่อนไขเพื่อทำงานเฉพาะในส่วนที่ตรงตามเงื่อนไขที่กำหนด

- `if (Condition) TrueCase;`
- `if (Condition) {TrueCase} else {FalseCase};`
- `if (Condition1) {TrueCase1}`
`else if (Condition2){TrueCase2}`
`else {FalseCase};`

```
var a = 3;  
if (a>1) {  
  x = 'yes';  
}  
else {  
  x = 'no';  
}
```

Conditional (Ternary) Operator

การเขียน if else แบบสั้น เหมาะสำหรับคืนค่าให้กับตัวแปร

`Condition ? TrueCase : FalseCase`

```
var a = 3;  
var x = a>1 ? 'yes' : 'no';  
// x is 'yes';
```

switch

การพิสูจน์เงื่อนไขเพื่อทำงานเฉพาะในส่วนที่ตัวแปรตรงตามค่าที่กำหนด

```
var a = 3;  
switch (a){  
  case 1: x='one'; break;  
  case 2: x='two'; break;  
  case 3: x='three'; break;  
  default: x='others';  
} // x is 'three';
```



Function

JavaScript offers a variety of function types, each with its own strengths and purpose.

1. Declaration

```
function square(x) { return x * x; }
```

2. Expression

```
const square = function(x) { return x * x; };
```

3. Constructor

```
function Person(name) { this.name = name; }  
const john = new Person("John");
```

ES6 4. Arrow

```
const square = x => x*x;
```

ES6 5. Generator

```
function* fibonacci(n){ yield ...}
```

6. Promise

ES6 7. Async/Await

Declaration Example

```
function a(){ /*do something*/ }
```

สร้างฟังก์ชันที่ไม่ต้องการพารามิเตอร์ และไม่มีการคืนค่า

```
function a(x,y) { /*do something*/ }
```

สร้างฟังก์ชันที่ต้องกำหนดพารามิเตอร์ x และ y ซึ่งไม่มีการคืนค่า

```
function a(){ /*do something*/ return b; }
```

สร้างฟังก์ชันที่ไม่ต้องการพารามิเตอร์ และคืนค่า b;

```
function a(x,y) { /*do something*/ return b; }
```

สร้างฟังก์ชันที่ต้องกำหนดพารามิเตอร์ x และ y และคืนค่า b



Function Arguments

Arguments in JavaScript play a crucial role in passing information to functions and controlling their behavior. Here's a breakdown of their key aspects:

Classical JavaScript

ปกติฟังก์ชันจะถูกออกแบบให้มีการระบุพารามิเตอร์แบบเรียงลำดับซึ่งสามารถระบุชื่อเพื่อเรียกใช้งานประมวลผลภายใน

```
function Sum(a,b){  
    return a+b;  
}  
var x = Sum(1, 2); // x is 3
```

พารามิเตอร์ทั้งหมดที่ส่งผ่านมายังฟังก์ชันสามารถอ้างอิงได้ด้วย arguments ซึ่งมีลักษณะคล้าย array

```
function Sum(a,b){  
    return arguments[0] + arguments[1];  
}  
var x = Sum(1, 2); // x is 3
```

GEE

สามารถใส่เป็น dictionary (JSON) เพียงค่าเดียวที่มีหลาย key และ value ซึ่งทำให้ไม่จำเป็นต้องเรียงลำดับกันก็ได้ โดยชื่อพารามิเตอร์ให้อ้างอิงชื่อตามคู่มือที่ method นั้นได้ออกแบบไว้

```
var x = eeObject.Sum({b: 2, a: 1}); // x is 3
```

เบื้องหลังการทำงาน

```
function method(a, b){  
    if (arguments.length==1 && arguments[0] instanceof Object){  
        var a =arguments[0].a;  
        var b =arguments[0].b;  
    }  
    return a+b;  
}
```



By Value & By Pointer Arguments

หากค่าให้กับฟังก์ชันแบ่งได้เป็นสองประเภทหลัก คือ by value และ by pointer โดยภาษา JavaScript จะส่งค่าแบบ by value กับตัวแปรประเภท primitive type เช่น number, string และ Boolean และแบบ by pointer สำหรับตัวแปรประเภท object

By Value

เป็นการคัดลอกค่าของตัวแปรต้นฉบับไปยังตัวแปรที่รับค่าในฟังก์ชัน การเปลี่ยนแปลงค่าของตัวแปรรับค่าในฟังก์ชันจะไม่ส่งผลต่อค่าของตัวแปรต้นฉบับ

```
var x = 10;  
function fv(a) {  
    a = 20;  
}  
fv(x);  
// x is 10
```

By Pointer

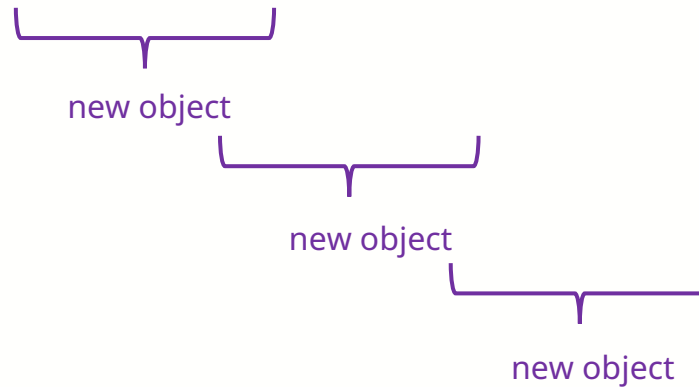
เป็นการอ้างอิงไปยังตัวแปรต้นฉบับในหน่วยความจำ การเปลี่ยนแปลงค่าของตัวแปรรับค่าในฟังก์ชันจะส่งผลต่อค่าของตัวแปรต้นฉบับด้วย

```
var y = {a:10};  
function fp(a) {  
    a.a = 20;  
}  
fp(y);  
// y is {a:20}
```

Chained Method

การเรียกใช้งานพฤติกรรม (method หรือ function) ของ object ในลักษณะ Chaining

```
a.method1().method2().method3();
```



ถึงแม้ว่าภาษา JavaScript เป็นแบบไม่สนใจการเว้นวรรค ขึ้นบรรทัดใหม่ และย่อหน้า แต่การทำงานที่ซับซ้อนก็ควรเขียนในลักษณะแบ่งเป็นย่อหน้า เพื่อให้อ่านเข้าใจได้ง่าย

```
a.method1()  
  .method2()  
  .method3();
```

การเขียนแบบเต็มรูปแบบในลักษณะประกาศตัวแปร

```
var x = a.method1();  
var y = x.method2();  
var z = y.method3();
```



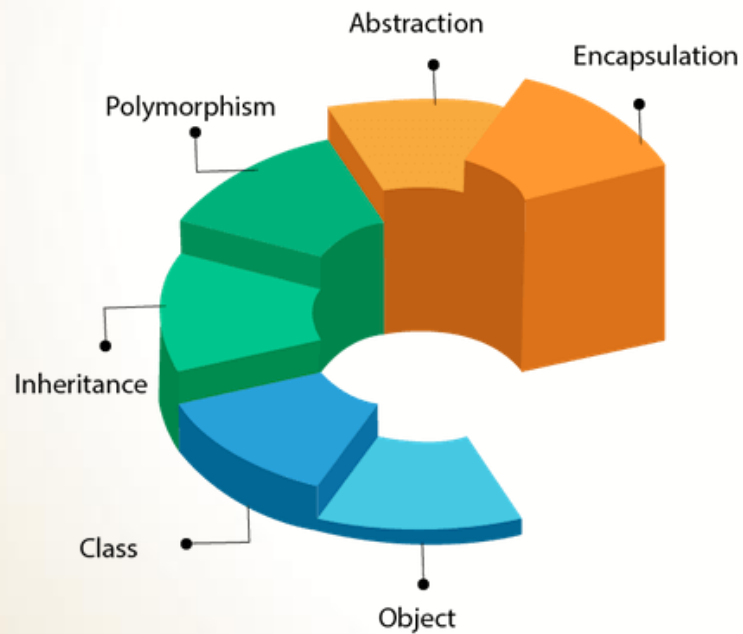
OOP

Object Oriented Programming



Object-Oriented Programming (OOP)

In simple terms, a variable in JavaScript is like a named container that stores information. You can use variables to hold different types of data, like numbers, text, booleans (true/false), arrays, or even objects.



- Objects: Bundles of related data (properties) and functions (methods) that represent real-world entities.
- Classes: Templates for creating multiple objects with similar properties and methods. JavaScript uses prototype-based inheritance, where objects inherit properties and methods from other objects instead of classes.
- Encapsulation: Grouping data and functions together within an object, hiding internal details and restricting access to protect data integrity.
- Inheritance: Creating new objects based on existing ones, inheriting their properties and methods while allowing for customization.
- Polymorphism: Different objects responding to the same message (method call) in different ways depending on their type.

Class vs Instance

Class / Constructor

Class หรือ constructor คือ ต้นแบบวัตถุ ที่มีการกำหนด property และ method ไว้

ES5

```
function A(){  
  //methods  
  //properties  
  //events  
}
```

ES6

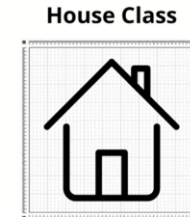
```
class A{  
  //methods  
  //properties  
  //events  
}
```

Instance / Object

Instance หรือ Object คือวัตถุที่ถูกสร้างจากต้นแบบจะมี property และ method ตามที่ได้กำหนดไว้

การสร้าง object จาก class ด้วย new Class()

```
var a = new A();
```



House Instance



House Instance



GEE ไม่ใช่ new ในการสร้าง instance เนื่องจากสั่งงานผ่าน API ไปสร้าง instance ที่ server ไม่ได้ถูกสร้างอยู่ที่ code editor หรือ client script การสั่งให้สร้าง instance object จึงเป็น method เช่น ee.Image() เป็นคำสั่งให้สร้าง instance ee.Image



Object

Objects are fundamental building blocks in JavaScript, serving as containers for data and functionality. They allow you to group related properties and methods together, representing real-world entities like users, products, or even shapes. Here's a breakdown of objects in JavaScript:

Properties: `object.property`

- Key-value pairs where the key identifies the data and the value holds the actual data.
- Can store various data types like numbers, strings, Booleans, arrays, other objects, and even functions.
- Accessed using the dot notation.

Methods: `object.method()` หรือ `object.method(params);`

- Functions defined within an object, representing its actions or behaviors.
- Can access and modify the object's properties.
- Invoked using the dot notation followed by parentheses and optional arguments.



Array vs Dictionary

methods / properties

Name	Name
at()	keys()
concat()	lastIndexOf()
constructor	length
copyWithin()	map()
entries()	pop()
every()	prototype
fill()	push()
filter()	reduce()
find()	reduceRight()
findIndex()	reverse()
flat()	shift()
flatMap()	slice()
forEach()	some()
from()	sort()
includes()	splice()
indexOf()	toString()
isArray()	unshift()
join()	valueOf()

Array

Array หรือ List เป็น Indexed collections ประเภทหนึ่งที่พื้นฐานที่สุดของเกือบทุกภาษา

```
var a = new Array();
```

หรือ

```
var a = [ ];
```

```
a.add(1);  
console.log(a[0]); // 1
```

ระบุ index เริ่มจาก 0 เพื่อเข้าถึงข้อมูล

https://www.w3schools.com/jsref/jsref_obj_object.asp

https://www.w3schools.com/jsref/jsref_obj_array.asp

Dictionary

Object หรือ Dictionary เป็น Keys collections ประเภทหนึ่งที่พื้นฐานที่สุดของเกือบทุกภาษา

```
var a = new Object();
```

หรือ

```
var a = { };
```

```
a['first'] = 1;  
console.log(a['first']); // 1
```

ระบุ key ที่อยู่ในรายการเพื่อเข้าถึงข้อมูล

[OneCompiler https://onecompiler.com/nodejs/3zwppct4h](https://onecompiler.com/nodejs/3zwppct4h)

methods

Name
constructor
keys()
prototype
toString()
valueOf()

properties

แรกเริ่ม instance object จะไม่มี properties แต่ property จะเพิ่มขึ้นตามที่กำหนด key และ value ให้กับ object

`object[key] = value`



Dictionary Prototype Methods

Dictionary ไม่มี prototype property และ methods
การทำงานกับ Dictionary ควรแปลงเป็น Array เสียก่อน มีหลายวิธีเช่น

Object.keys(var)

ใช้ Class Object หา keys ของตัวแปรนั้น

```
var a = {x: 1, y:2};  
var k = Object.keys(a);  
console.log(k); // x y
```

Object.values(var)

ใช้ Class Object หา values ของตัวแปรนั้น

```
var a = {x: 1, y:2};  
var v = Object.values(a);  
console.log(v); // 1 2
```

Object.entries(var)

ใช้ Class Object หา values ของตัวแปรนั้น

```
var a = {x:1, y:2};  
for (var i of Object.entries(a)){  
    console.log(i);  
}  
// [x, 1] [y, 2]
```



Object as Dictionary Instance

Everything not primitive in JavaScript is an Object, and basic object is dictionary.

Key / Property

กำหนด property ให้กับ object แบบ key

```
a = { };  
a['first'] = 1;  
console.log(a['first']); // 1  
console.log(a); // {first:1}
```

กำหนด property ให้กับ object แบบ property

```
a = { };  
a.first = 1;  
console.log(a.first); // 1  
console.log(a); // {first:1}
```

Method

เนื่องจาก property ของ object สามารถเป็น primitive value หรือ object อะไรก็ได้ ดังนั้นเราสามารถกำหนด method หรือ function ให้กับ object ได้อีกด้วย

```
a = { };  
a.sum = function(x,y){  
    return x+y;  
}  
console.log(a.sum(1,2));  
// 3
```

การกำหนด property ให้กับ instance object จะมีผลกับ object นั้นเพียงตัวเดียวและจะไม่ส่งผลกระทบต่อโครงสร้างของ object อื่นๆที่สร้างจาก class เดียวกัน

ดังนั้น หากต้องการให้เป็นต้นแบบกับทุก object ที่จะสร้างต้องกำหนดให้กับระดับ class นั่นคือ `Object.prototype.property`

ภาษา JavaScript ไม่สามารถกำหนดรูปแบบการอ่าน (get) และเขียน (set) ค่าให้กับ property รวมถึง method ได้ ดังนั้นจึงไม่นิยมใช้ property แต่จะใช้ method ตั้งชื่อในลักษณะ `getProperty()` และ `setProperty(value)` แทนเพื่อความชัดเจนในการใช้งาน



Array Prototype Methods

Array methods ที่จำเป็นต้องรู้

sort

เรียงลำดับรายการจากน้อยไปมาก

```
var a = [5,1,3];  
var b = a.sort();  
console.log(a); //[1,3,5]  
console.log(b); //[1,3,5]
```

includes

หาลำดับที่ของรายการที่ระบุ

```
var e = [5,1,3];  
console.log(e.includes(1)); //true  
console.log(1 in e); //true
```

find / findIndex

ค้นหารายการใน Array เพียงตัวเดียวตามเงื่อนไข

```
var h = [5,1,3,1,9];  
console.log(h.find(  
function(x){return x>2})); //5  
console.log(h.findIndex(  
function(x){return x>2})); //0
```

reverse

เรียงลำดับรายการจากมากไปน้อย

```
var c = [5,1,3];  
var d = c.reverse();  
console.log(c); // [5,3,1]  
console.log(d); // [5,3,1]
```

indexOf

```
var f = [5,1,3,1,9];  
console.log(e.indexOf(1)); //1
```

lastIndexOf

```
var f = [5,1,3,1,9];  
console.log(e.lastIndexOf(1)); //3
```


Array Prototype Methods

Array methods ที่จำเป็นต้องรู้

at

รับค่าจากรายการที่ระบุ (นับเริ่มจาก 0)

```
var a = [1,2,3];  
console.log(a.at(1));  
//2
```

join

เชื่อมทุกรายการใน Array ด้วย

```
var c = [1,2,3];  
console.log(b.join('-'));  
//1-2-3
```

pop

ดึงข้อมูลออกจากรายการท้ายสุด

```
var e = [1,2,3];  
var f = e.pop();  
console.log(e);  
console.log(f);
```

concat

เชื่อม 2 Array เข้าด้วยกัน

```
var i = [1,2,3];  
var j = [4,5,6];  
console.log(i.concat(j));  
//[1,2,3,4,5,6]
```

slice

เลือกช่วง Array

```
var b = [1,2,3];  
console.log(b.slice(1,2));  
//[2]
```

unshift

แทรกข้อมูลเข้าไปในส่วนต้นของรายการ เอาข้อมูลออกจากรายการแรก

```
var d = [1,2,3];  
var e = d.unshift(-1,0);  
console.log(d); // [-  
1,0,1,2,3]  
console.log(e); // 5 count
```

shift

```
var g = [1,2,3];  
var h = g.shift();  
console.log(g);  
console.log(h);
```

flat

แปลง Array ให้เป็น 1D

```
var k = [[1,2,3],[4,5,6]];  
console.log(k.flat());  
//[1,2,3,4,5,6]
```



Array Prototype Methods

Array methods ที่จำเป็นต้องรู้

every

พิสูจน์ว่าใน Array ว่าตรงกับเงื่อนไขทุก

```
var c = [1,2,3];var fc =  
function(x){return x>0};var d =  
c.every(fc);console.log(d);
```

forEach

นำแต่ละรายการใน Array มาใช้กับฟังก์ชัน

```
var a = [1,2,3];  
var b = [];  
var fa = function(x){ b.push(x+1)};  
a.forEach(fa);  
console.log(b);
```

map

เปลี่ยนทุกรายการเป็นค่าใหม่โดยใช้ฟังก์ชันที่กำหนด

```
var m = [1,2,3];  
var fm = function(x){return x+1};  
console.log(m.map(fm));
```

some

พิสูจน์ว่าใน Array ว่าตรงกับเงื่อนไขบางรายการ

```
var c = [1,2,3];var fc =  
function(x){return x>0};var d =  
c.some(fc);console.log(d);
```

entries

สร้าง Iterator คู่ index และ value แต่ละรายการ

```
var d1 = [1,2,3].entries();  
do { var d2 = d1.next().value;  
console.log(d2);}  
while (d2);
```

ES6 for of

reduce

ลดรูปทุกรายการโดยใช้ฟังก์ชันลดรูป

```
var r = [1,2,3];  
var fr = function(acc, init)  
{return acc + init};  
console.log(r.reduce(fr));
```

AJAX and API

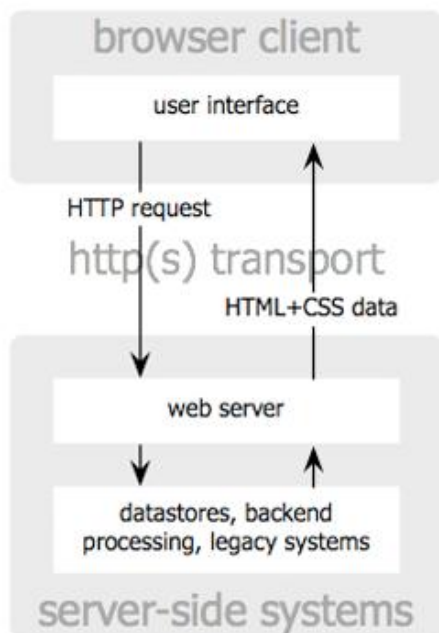
GEE JavaScript

```
var a = ee.Image (...);
var b = ee.Feature (...);
```

การสร้าง instance object ของ **GEE** ไม่ใช้ **new** เนื่องจากไม่ได้สร้างไว้จริงที่ client หรือเครื่องที่กำหลังเขียนโค้ดอยู่ แต่สั่งให้ GEE ไปสร้างไว้ที่ server แล้วเก็บไว้เป็นตัวแทน

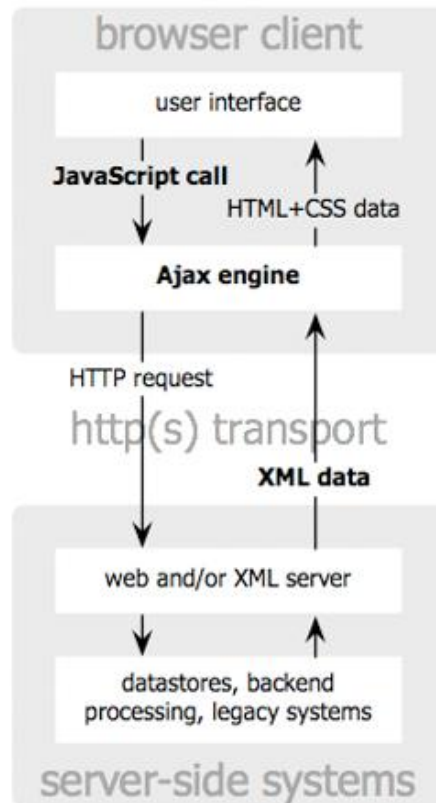
และโดยส่วนใหญ่ GEE Object จะไม่มี property แท้เนื่องจากต้องสั่งการให้ server ประมวลผล ดังนั้นจึงมีแต่ method หรือ function และ event เท่านั้น โดย property จะใช้แบบ `getProperty(...)` และ `setProperty(...)` เพื่อการคืนค่าละตั้งค่า

`Object.getInfo()` จะเป็นการแปลง Server Object ให้เป็น Client Object ซึ่งใช้เวลาในการประมวลผล และเป็นได้ทั้งแบบ `synchronize` ที่ต้องรอจนได้ข้อมูลจาก server จึงจะทำงานต่อไปได้ และ `asynchronize` ที่ประมวลผลเสร็จส่งไปที่ฟังก์ชัน `callback` ทำงานต่อไป (ถ้าไม่มีความจำเป็นที่จะใช้การทำงานบน Client จึงไม่ควรใช้ `getInfo()`)



classic
web application model

Jesse James Garrett / adaptivepath.com



Ajax
web application model



JavaScript Object Notation (JSON)

JSON เป็นข้อมูลของ JavaScript ที่เก็บในรูปแบบ Text (Notation) ในลักษณะ Object (Dictionary) และ Array (List)

JSON stands for JavaScript Object Notation. It's a popular data format used for data interchange and storage. Here's a breakdown of what it is and why it's so important:

Why use JSON?

- Data exchange: It's an easy and convenient way to exchange data between different applications and programming languages.
- Web development: It's widely used in web development for sending data between client and server, such as APIs and web applications.
- Configuration files: It's often used for storing configuration settings for applications.
- Data storage: It's a popular choice for storing structured data in files or databases.

Key features of JSON:

- Key-value pairs: Data is stored in pairs, with a key identifying the data and a value holding the actual data.
- Arrays: Ordered lists of values, similar to lists in other languages.
- Simple syntax: Easy to understand and write, making it accessible even for non-programmers.
- Human-readable: The text-based format allows for easy inspection and modification of data.



JavaScript Object Notation (JSON)

JSON (Text)

```
[{  
  a: 1,  
  b: 2,  
  c: 3,  
}]
```

HTTPRequest



HTTPResponse

Object

```
var json = '{"a":1, "b":2}';  
var obj = JSON.parse(json);  
console.log(obj); // {a:1, b:2}  
console.log(obj.a); //1
```

CSS

Cascading Style Sheets



Using CSS

- Internal - by using a `<style>` element in the `<head>` section
- External - by using a `<link>` element to link to an external CSS file
- Inline - by using the `style` attribute inside HTML elements

```
<html>
  <head>
    <style>
      tag { color: 'red'}
      #id { color: 'red'}
      .class { color: 'red'}
    </style>
    <link rel="stylesheet" href="external.css">
  </head>
  <body>
    <div style="color: 'red' " id="myDiv"> ..... </div>
  </body>
</html>
```

} Internal

} External

} Inline



HTML5 Tag

คุณสมบัติ	 div	 canvas
จุดประสงค์	จัดระเบียบเนื้อหาและสร้างองค์ประกอบที่มีสไตล์	วาดภาพและสร้างกราฟิก 2D และ 3D
คุณสมบัติหรือวิธีการ	ไม่มีคุณสมบัติหรือวิธีการเฉพาะใดๆ ที่ออกแบบมาสำหรับการวาดภาพหรือสร้างกราฟิก	มีวิธีการมากมายสำหรับการวาดภาพและสร้างกราฟิก
การใช้งานทั่วไป	ใช้ในการจัดระเบียบเนื้อหาบนหน้าเว็บ การสร้างองค์ประกอบที่มีสไตล์ และการสร้างองค์ประกอบแบบโต้ตอบ	ใช้ในการวาดภาพ เส้น รูปร่าง ข้อความ แอนิเมชัน และเกม



SVG: Scalable Vector Graphics ภาพกราฟิกส์เวกเตอร์ปรับขนาดได้) เป็นภาษามาร์กอัปบนมาตรฐาน XML สำหรับอธิบายภาพกราฟิกส์เวกเตอร์ 2 มิติ ทั้งที่เป็นภาพนิ่งและภาพเคลื่อนไหว เอสวีจีเป็นมาตรฐานเปิดที่ดูแลโดย เวิลด์ไวด์เว็บคอนซอร์เทียม เวอร์ชันล่าสุดคือ SVG 1.1

Color

การระบุสีสามารถระบุได้ 4 แบบคือ ใช้ชื่อสี (color name) เลขฐานสิหก (Hex RGB) เลขฐานสิบ (Dec RGB) และ HSL

Basic Colors

Named	Numeric	Color name	Hex rgb	Decimal
		black	#000000	0,0,0
		silver	#C0C0C0	192,192,192
		gray	#808080	128,128,128
		white	#FFFFFF	255,255,255
		maroon	#800000	128,0,0
		red	#FF0000	255,0,0
		purple	#800080	128,0,128
		fuchsia	#FF00FF	255,0,255
		green	#008000	0,128,0
		lime	#00FF00	0,255,0
		olive	#808000	128,128,0
		yellow	#FFFF00	255,255,0
		navy	#000080	0,0,128
		blue	#0000FF	0,0,255
		teal	#008080	0,128,128
		aqua	#00FFFF	0,255,255

Extended colors

Named	Numeric	Color name	Hex rgb	Decimal
		aliceblue	#f0f8ff	240,248,255
		antiquewhite	#faebd7	250,235,215
		aqua	#00ffff	0,255,255
		aquamarine	#7fffd4	127,255,212
		azure	#f0ffff	240,255,255
		beige	#f5f5dc	245,245,220
		bisque	#ffe4c4	255,228,196
		black	#000000	0,0,0
		blanchedalmond	#ffebcd	255,235,205
		blue	#0000ff	0,0,255
		blueviolet	#8a2be2	138,43,226
		brown	#a52a2a	165,42,42
		burlywood	#deb887	222,184,135
		cadetblue	#5f9ea0	95,158,160
		chartreuse	#7fff00	127,255,0
		chocolate	#d2691e	210,105,30
		coral	#ff7f50	255,127,80
		cornflowerblue	#6495ed	100,149,237
		cornsilk	#fff8dc	255,248,220
		crimson	#dc143c	220,20,60

		crimson	#dc143c	220,20,60
		cyan	#00ffff	0,255,255
		darkblue	#00008b	0,0,139
		darkcyan	#008b8b	0,139,139
		darkgoldenrod	#b8860b	184,134,11
		darkgray	#a9a9a9	169,169,169
		darkgreen	#006400	0,100,0
		darkgrey	#a9a9a9	169,169,169
		darkkhaki	#bdb76b	189,183,107
		darkmagenta	#8b008b	139,0,139
		darkolivegreen	#556b2f	85,107,47
		darkorange	#ff8c00	255,140,0
		darkorchid	#9932cc	153,50,204
		darkred	#8b0000	139,0,0
		darksalmon	#e9967a	233,150,122
		darkseagreen	#8fbc8f	143,188,143
		darkslateblue	#483d8b	72,61,139

		darkslategray	#2f4f4f	47,79,79
		darkslategrey	#2f4f4f	47,79,79
		darkturquoise	#00ced1	0,206,209
		darkviolet	#9400d3	148,0,211
		deeppink	#ff1493	255,20,147
		deepskyblue	#00bfff	0,191,255
		dimgray	#696969	105,105,105
		dimgrey	#696969	105,105,105
		dodgerblue	#1e90ff	30,144,255
		firebrick	#b22222	178,34,34
		floralwhite	#fffaf0	255,250,240
		forestgreen	#228b22	34,139,34
		fuchsia	#ff00ff	255,0,255
		gainsboro	#dcdcdc	220,220,220
		ghostwhite	#f8f8ff	248,248,255
		gold	#ffd700	255,215,0
		goldenrod	#daa520	218,165,32
		gray	#808080	128,128,128
		green	#008000	0,128,0



CSS Value

CSS can be used in two part, general CSS and JavaScript. According to JavaScript cannot be use special character (-) in the code, the form of CSS as object property must be remove '-' from the property name and start the second character part with upper case.

Example	CSS Style	JavaScript Property
font color	color: #000000;	div.color = '#000000';
background color	background-color: #000000;	div.backgroundColor = '#000000';
outline with	outline-with: 2px;	div.outlineWith = '2px';



CSS in GEE

GEE is using CSS in particular part, e.g. `ee.FeatureCollection.style`, `ee.ImageCollection.style`, `ui.Panel.style`

`Map.addLayer(eeObject, visParams, name, shown, opacity)`

`ee.FeatureCollection.draw` 

[Send feedback](#)

Paints a vector collection for visualization. Not intended for use as input to other algorithms.

Usage		Returns
<code>FeatureCollection.draw(color, pointRadius, strokeWidth)</code>		Image
Argument	Type	Details
this: collection	FeatureCollection	The collection to draw.
color	String	A hex string in the format RRGGBB specifying the color to use for drawing the features.
pointRadius	Integer, default: 3	The radius in pixels of the point markers.
strokeWidth	Integer, default: 2	The width in pixels of lines and polygon borders.

`ee.ImageCollection.draw` 

[Send feedback](#)

Paints a vector collection for visualization. Not intended for use as input to other algorithms.

Usage		Returns
<code>ImageCollection.draw(color, pointRadius, strokeWidth)</code>		Image
Argument	Type	Details
this: collection	FeatureCollection	The collection to draw.
color	String	A hex string in the format RRGGBB specifying the color to use for drawing the features.
pointRadius	Integer, default: 3	The radius in pixels of the point markers.
strokeWidth	Integer, default: 2	The width in pixels of lines and polygon borders.

<https://developers.google.com/earth-engine/apidocs/map-addlayer>

<https://developers.google.com/earth-engine/apidocs/ee-data-getmapid>

<https://developers.google.com/earth-engine/apidocs/ee-featurecollection-draw>

<https://developers.google.com/earth-engine/apidocs/ee-imagecollection-draw>



CSS in GEE

GEE is using CSS in particular part, e.g. `ee.FeatureCollection.style`, `ee.ImageCollection.style`, `ui.Panel.style`

ee.FeatureCollection.style

[Send feedback](#)

Draw a vector collection for visualization using a simple style language.

Usage	Returns
<code>FeatureCollection.style(color, pointSize, pointShape, width, fillColor, styleProperty, neighborhood, lineType)</code>	Image

Argument	Type	Details
this: collection	FeatureCollection	The collection to draw.
color	String, default: "black"	A default color (CSS 3.0 color value e.g. 'FF0000' or 'red') to use for drawing the features. Supports opacity (e.g.: 'FF000088' for 50% transparent red).
pointSize	Integer, default: 3	The default size in pixels of the point markers.
pointShape	String, default: "circle"	The default shape of the marker to draw at each point location. One of: 'circle', 'square', 'diamond', 'cross', 'plus', 'pentagram', 'hexagram', 'triangle', 'triangle_up', 'triangle_down', 'triangle_left', 'triangle_right', 'pentagon', 'hexagon', 'star5', 'star6'. This argument also supports the following Matlab marker abbreviations: 'o', 's', 'd', 'x', '+', 'p', 'h', 'A', 'v', '<', '>'.
width	Float, default: 2	The default line width for lines and outlines for polygons and point shapes.
fillColor	String, default: null	The color for filling polygons and point shapes. Defaults to 'color' at 0.66 opacity.
styleProperty	String, default: null	A per-feature property expected to contain a dictionary. Values in the dictionary override any default values for that feature.
neighborhood	Integer, default: 5	If styleProperty is used and any feature has a pointSize or width larger than the defaults, tiling artifacts can occur. Specifies the maximum neighborhood (pointSize + width) needed for any feature.
lineType	String, default: "solid"	The default line style for lines and outlines of polygons and point shapes. Defaults to 'solid'. One of: solid, dotted, dashed.

ee.ImageCollection.style

[Send feedback](#)

Draw a vector collection for visualization using a simple style language.

Usage	Returns
<code>ImageCollection.style(color, pointSize, pointShape, width, fillColor, styleProperty, neighborhood, lineType)</code>	Image

Argument	Type	Details
this: collection	FeatureCollection	The collection to draw.
color	String, default: "black"	A default color (CSS 3.0 color value e.g. 'FF0000' or 'red') to use for drawing the features. Supports opacity (e.g.: 'FF000088' for 50% transparent red).
pointSize	Integer, default: 3	The default size in pixels of the point markers.
pointShape	String, default: "circle"	The default shape of the marker to draw at each point location. One of: 'circle', 'square', 'diamond', 'cross', 'plus', 'pentagram', 'hexagram', 'triangle', 'triangle_up', 'triangle_down', 'triangle_left', 'triangle_right', 'pentagon', 'hexagon', 'star5', 'star6'. This argument also supports the following Matlab marker abbreviations: 'o', 's', 'd', 'x', '+', 'p', 'h', 'A', 'v', '<', '>'.
width	Float, default: 2	The default line width for lines and outlines for polygons and point shapes.
fillColor	String, default: null	The color for filling polygons and point shapes. Defaults to 'color' at 0.66 opacity.
styleProperty	String, default: null	A per-feature property expected to contain a dictionary. Values in the dictionary override any default values for that feature.
neighborhood	Integer, default: 5	If styleProperty is used and any feature has a pointSize or width larger than the defaults, tiling artifacts can occur. Specifies the maximum neighborhood (pointSize + width) needed for any feature.
lineType	String, default: "solid"	The default line style for lines and outlines of polygons and point shapes. Defaults to 'solid'. One of: solid, dotted, dashed.

<https://developers.google.com/earth-engine/apidocs/ee-featurecollection-style>



CSS in GEE

GEE is using CSS in particular part, e.g. [ee.FeatureCollection.style](#), [ee.ImageCollection.style](#), [ui.Panel.style](#)

ui.Panel.style

Returns the widget's style ActiveDictionary, which can be modified to update the widget's styles.

Properties which behave like their CSS counterparts:

- height, maxHeight, minHeight (e.g. '100px')
- width, maxWidth, minWidth (e.g. '100px')
- padding, margin (e.g. '4px 4px 4px 4px' or simply '4px')
- color, backgroundColor (e.g. 'red' or '#FF0000')
- border (e.g. '1px solid black')
- fontSize (e.g. '24px')
- fontWeight (e.g. 'bold' or '100')
- fontFamily (e.g. 'monospace' or 'serif')
- textAlign (e.g. 'left' or 'center')
- textDecoration (e.g. 'underline' or 'line-through')
- whiteSpace (e.g. 'nowrap' or 'pre')
- shown (true or false)

Supported custom layout properties (see ui.Panel.Layout documentation):

- stretch ('horizontal', 'vertical', 'both')
- position ('top-right', 'top-center', 'top-left', 'bottom-right', ...)

ui.Thumbnail.style

[Send feedback](#)

Returns the widget's style ActiveDictionary, which can be modified to update the widget's styles.

Properties which behave like their CSS counterparts:

- height, maxHeight, minHeight (e.g. '100px')
- width, maxWidth, minWidth (e.g. '100px')
- padding, margin (e.g. '4px 4px 4px 4px' or simply '4px')
- color, backgroundColor (e.g. 'red' or '#FF0000')
- border (e.g. '1px solid black')
- fontSize (e.g. '24px')
- fontWeight (e.g. 'bold' or '100')
- fontFamily (e.g. 'monospace' or 'serif')
- textAlign (e.g. 'left' or 'center')
- textDecoration (e.g. 'underline' or 'line-through')
- whiteSpace (e.g. 'nowrap' or 'pre')
- shown (true or false)

Supported custom layout properties (see ui.Panel.Layout documentation):

- stretch ('horizontal', 'vertical', 'both')
- position ('top-right', 'top-center', 'top-left', 'bottom-right', ...)



THANK YOU

QUIZ

<https://forms.gle/2CJCus5tzKx7ufK86>

